

Well-Founded Recursion and Proof Automation

Lecture 23

Section 1

Part I: The Problem

What we left unfinished

In Lecture 21 we defined a function `buildTree` for Huffman coding. It took a list of weighted trees and merged them until one remained. The recursive call operated on a *shorter* list, but not on a *subterm* of the input.

Lean accepted the definition because it could infer a **well-founded** termination argument: the length of the list decreases. But we never explained what that means, and we used `decreasing_by` without understanding what it does.

This lecture explains what well-founded recursion is, how Lean uses it, and how to control it with `termination_by` and `decreasing_by`. We then cover the automation tactics — `simp`, `omega`, `decide` — that make proofs less tedious.

Two kinds of recursion in Lean

Structural recursion: the recursive call is on a direct subterm of a constructor argument. Lean can verify termination by checking that the recursion follows the inductive structure.

```
def length : List  $\alpha$  → Nat
| []      => 0
| _ :: t => 1 + length t
```

Here t is a subterm of the input $_ :: t$. No extra annotation needed.

Well-founded recursion: the recursive call is not on a subterm, but some *measure* of the arguments decreases. Lean needs help identifying what decreases and why.

A motivating example: GCD

The Euclidean algorithm does not recur on a subterm:

```
def gcd (m n : Nat) : Nat :=  
  if h : m = 0 then n  
  else gcd (n % m) m
```

The recursive call passes $n \% m$ and m . Neither is a subterm of the original arguments. But $n \% m < m$ whenever $m \neq 0$, so the first argument *decreases*. We tell Lean:

```
termination_by m  
decreasing_by exact Nat.mod_lt n (Nat.pos_of_ne_zero h)
```

Note the dependent `if h : —` this binds $h : m \neq 0$ in the else branch, which the termination proof needs.

Another example: division by repeated subtraction

```
def div (n k : Nat) : Nat :=  
  if k = 0 then 0  
  else if n < k then 0  
  else 1 + div (n - k) k  
termination_by n
```

Here $n - k < n$ whenever $k > 0$ and $n \geq k$. Lean uses n as the measure. The recursive call decreases n to $n - k$, which is strictly less.

Without the `termination_by` clause, Lean tries to infer the measure automatically. If it fails, it reports an error and suggests you add one.

The Ackermann function: lexicographic measures

Some functions need more than a single decreasing number. The Ackermann function:

```
def ack (n m : Nat) : Nat :=  
  match n, m with  
  | 0,      m      => m + 1  
  | n + 1, 0      => ack n 1  
  | n + 1, m + 1 => ack n (ack (n + 1) m)  
termination_by (n, m)
```

The measure is the *pair* (n, m) , ordered lexicographically: $(a, b) < (c, d)$ iff $a < c$, or $a = c$ and $b < d$. Let us check each recursive call.

Ackermann: checking the three calls

Case	Call	Measure	Why it decreases
$\text{ack } (n+1) \ 0$	$\text{ack } n \ 1$	$(n, 1)$	$n < n+1$
$\text{ack } (n+1) \ (m+1)$	inner: $\text{ack } (n+1) \ m$	$(n+1, m)$	$m < m+1$
$\text{ack } (n+1) \ (m+1)$	outer: $\text{ack } n \ (\dots)$	$(n, ?)$	$n < n+1$

The interesting case is the outer call in the third case. Its second argument is $\text{ack } (n+1) \ m$, which is a *huge* number. But lexicographic order does not care: since $n < n+1$, we have $(n, \text{anything}) < (n+1, m+1)$. The first component alone settles it.

Why lexicographic order matters

This is exactly why we need the lexicographic order rather than, say, $n + m$ as a measure. In the outer call of case 3, n decreases but the second argument *explodes* (ack 3 3 = 61, ack 4 1 = 65533). No single number $n + m$ decreases.

The lexicographic order on $\text{Nat} \times \text{Nat}$ is well-founded because:

- $<$ on Nat is well-founded (no infinite descending chains).
- The lex product of two well-founded relations is well-founded.

Lean knows this: the default `WellFoundedRelation` instance for $(\alpha \times \beta)$ is the lexicographic order, derived from the instances for α and β via instance chaining (Lecture 22).

Section 2

Part II: The Theory

Why does `termination_by m` work?

We claimed that `termination_by m` proves `gcd` terminates because m decreases. But what does “decreases” mean precisely?

We need a relation $\prec$ on the measure's type with a special property: **you cannot keep going down forever**. If you start at any value and keep moving to smaller values, you must eventually stop. A relation with this property is called **well-founded**.

$<$ on `Nat` is well-founded: starting from any n , you can go $n > n - 1 > \dots > 1 > 0$ and then you are stuck — there is no natural number less than 0. Every descending chain is finite.

In contrast, $<$ on `Int` is *not* well-founded: $0 > -1 > -2 > \dots$ goes on forever. You could not use it as a termination measure.

Lean formalizes well-foundedness via a predicate `Acc`. An element x is **accessible** with respect to a relation r if every element below it is also accessible:

```
inductive Acc (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :  $\alpha \rightarrow \text{Prop}$  where  
  | intro (x :  $\alpha$ )  
    (h :  $\forall y, r\ y\ x \rightarrow \text{Acc}\ r\ y$ ) : Acc r x
```

This looks circular, but it is not. Consider $<$ on `Nat`: 0 is accessible vacuously (nothing is below it). Then 1 is accessible (the only thing below it is 0). Then 2, and so on.

Extracting from Acc

Since `Acc` has one constructor, we can pattern-match on it to extract the field $h : \forall y, r\ y\ x \rightarrow \text{Acc}\ r\ y$:

```
def Acc.inv {r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ } {x y :  $\alpha$ }  
  (hacc : Acc r x) (hr : r y x)  
  : Acc r y :=  
  match hacc with  
  | .intro _ h => h y hr
```

This says: if x is accessible and $y \prec x$, then y is accessible. The proof just unpacks `Acc.intro` — which packed a proof h that *every* predecessor of x is accessible — and applies h to y .

Proving $<$ is well-founded on Nat

We need to show $\forall n, \text{Acc } (<) n$. The proof is by induction:

```
theorem nat_lt_wf (n : Nat) : Acc ( $\cdot < \cdot$ ) n := by  
  induction n with  
  | zero =>  
    -- No m satisfies  $m < 0$ , so  $\forall$  is vacuous.  
    exact Acc.intro 0  
    (fun _ hm => absurd hm (Nat.not_lt_zero _))  
  | succ n ih =>  
    -- Any  $m < n+1$  is either  $m = n$  or  $m < n$ .  
    exact Acc.intro (n + 1) (fun m hm => by  
      have : m = n  $\vee$  m < n := by omega  
      cases this with  
      | inl h => subst h; exact ih  
      | inr h => exact ih.inv h)
```

Reading the proof

Base case: 0 is accessible because no m satisfies $m < 0$.

Successor case: we have $ih : \text{Acc}(<) n$ and must show $\text{Acc}(<) (n + 1)$. Take any $m < n + 1$. We need $\text{Acc}(<) m$. Now, Acc.inv lets us extract accessibility of predecessors: it has type $\text{Acc}(<) n \rightarrow m < n \rightarrow \text{Acc}(<) m$. But it requires $m < n$ **strictly** — and we only know $m < n + 1$, i.e. $m \leq n$. So we split:

- If $m < n$: apply Acc.inv to ih .
- If $m = n$: ih is literally $\text{Acc}(<) n = \text{Acc}(<) m$. Use it directly.

WellFounded and WellFoundedRelation

A relation r is **well-founded** if every element is accessible:

```
structure WellFounded (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) : Prop where  
  apply :  $\forall x, \text{Acc } r \ x$ 
```

(It is a structure with one field, `apply`. Given `wf : WellFounded r`, you write `wf.apply x` to get `Acc r x`.)

The standard library wraps this in a type class (Lecture 22):

```
class WellFoundedRelation ( $\alpha$  : Type) where  
  rel :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$   
  wf : WellFounded rel
```

The instance for `Nat` uses `<`. You can define other well-founded relations on `Nat` (for example, divisibility) — but `<` is the default.

Instances for WellFoundedRelation

For pairs $(\alpha \times \beta)$, the default WellFoundedRelation instance is the lexicographic order, built from the instances for α and β . This is why `termination_by n m => (n, m)` works for Ackermann.

To summarize the chain: you write `termination_by m`, Lean sees $m : \text{Nat}$, looks up `WellFoundedRelation Nat`, finds the instance with `rel = (<)`, and uses that relation to check that each recursive call decreases. For pairs, it finds the `lex` instance, which itself is built from the `Nat` instance — again instance chaining.

How Lean uses this

When you write a function with `termination_by expr`, Lean:

- 1 Looks up the `WellFoundedRelation` instance for the type of `expr` (e.g. `<` on `Nat`, lex order on `Nat × Nat`).
- 2 For each recursive call, generates a **proof obligation**: that `expr` at the recursive call is *less than* `expr` at the current call, according to that relation.
- 3 Tries to discharge these obligations automatically (using `simp` and `omega` internally).
- 4 If that fails, you supply your own proof with `decreasing_by`.

Note that the elaborator does all this work *before* sending the term to the kernel. The kernel receives a term that already contains the termination proofs — it does not know about `termination_by`.

decreasing_by: custom termination proofs

When Lean cannot discharge the proof obligation automatically, you write `decreasing_by` with a tactic proof:

```
def gcd (m n : Nat) : Nat :=  
  if h : m = 0 then n  
  else gcd (n % m) m  
termination_by m  
decreasing_by  
  apply Nat.mod_lt  
  exact Nat.pos_of_ne_zero h
```

The goal is $n \% m < m$. We prove it from $m \neq 0$. Note the `if h : m = 0`: the dependent `if` (from Lecture 22) binds the proof $h : m \neq 0$ in the `else` branch, which we use in `decreasing_by`.

Section 3

Part III: Proof Automation

The landscape of automation

So far we have written proofs step by step: `intro`, `cases`, `rw`, `exact`, `induction`. This works, but it is tedious for routine goals. Lean provides several automation tactics:

Tactic	Domain	Method
<code>simp</code>	equational rewriting	applies lemma database
<code>omega</code>	linear arithmetic over <code>Nat/Int</code>	decision procedure
<code>decide</code>	closed decidable propositions	evaluation
<code>positivity</code>	$0 \leq x, 0 < x$ goals	compositional analysis

Each has a different scope. We focus on the first three.

simp: the simplifier

`simp` repeatedly applies rewrite rules from a database until no more apply. Each rule is a lemma of the form $A = B$ or $A \leftrightarrow B$, applied left-to-right. For equalities ($A = B$), `simp` replaces occurrences of A with B . For iff's ($A \leftrightarrow B$), it does the same but for propositions — replacing a proposition A in the goal with the equivalent B :

```
-- Equality rule: Nat.zero_add : 0 + n = n
-- simp rewrites (0 + x) to x

-- Iff rule: Nat.lt_iff_add_one_le : m < n ↔ m + 1 ≤ n
-- simp rewrites (m < n) to (m + 1 ≤ n)
```

```
example : 0 + n + 0 = n := by simp
```

`simp` works bottom-up: it first simplifies subterms, then the whole expression. It keeps going until no rule applies.

Registering your own simp lemmas

Tag a theorem with `@[simp]` to add it to the database:

```
def double (n : Nat) := n + n

@[simp] theorem double_zero : double 0 = 0 := rfl
@[simp] theorem double_succ :
  double (n + 1) = double n + 2 := by omega
```

Now `simp` knows how to simplify `double`. But what about `double 3`? The `simp` lemmas handle `double 0` and the successor case, but `simp` does not unfold definitions by default. Writing `simp [double]` tells it to also unfold the *definition* of `double` — treating its defining equation as an additional rewrite rule.

Rule of thumb: a good `simp` lemma rewrites to something simpler. Bad lemmas (like $a + b = b + a$) cause infinite loops — `simp` would rewrite back and forth forever.

Variants of `simp`

`simp` has several important variants:

`simp only [h1, h2, ...]` — uses *only* the listed lemmas, not the full database. This makes proofs reproducible: they will not break when new `simp` lemmas are added elsewhere.

`simp [f]` — unfolds the definition of `f` and then simplifies. Useful when the goal involves a function you defined.

`simp_all` — simplifies both the goal and all hypotheses, using each hypothesis as a potential rewrite rule.

`simp?` — runs `simp` and then prints the minimal `simp only` call that achieves the same result. The recommended workflow: use `simp` while exploring, then run `simp?` and replace with `simp only` for the final proof.

simp with hypotheses

You can pass hypotheses to `simp`:

```
example (h : x = 0) : x + x = 0 := by  
  simp [h]
```

Here `simp` uses $h : x = 0$ as a rewrite rule, replacing x with 0 , and then simplifies $0 + 0 = 0$ using `Nat.add_zero`.

You can also simplify a hypothesis:

```
example (h : x + 0 = y) : x = y := by  
  simp at h  
  exact h
```

`simp at h` rewrites the hypothesis h (reducing $x + 0$ to x), giving $h : x = y$.

omega: linear arithmetic

Linear arithmetic means expressions built from variables, integer constants, addition, subtraction, and multiplication *by a constant* (like $3 \cdot n$) — but not variable-times-variable (like $n \cdot m$). omega is a decision procedure that can prove any true statement in this fragment:

```
example :  $\forall n : \text{Nat}, 0 \leq n := \text{by omega}$ 
```

```
example (h :  $n < m$ ) :  $n + 1 \leq m := \text{by omega}$ 
```

```
example (h :  $a + b = 10$ ) (h2 :  $a \leq 3$ )  
  :  $b \geq 7 := \text{by omega}$ 
```

omega is **complete** for its domain: if the goal is a true linear statement over Nat or Int, omega will prove it. It will fail on $n * m = m * n$ because that involves a product of two variables.

This is the tactic that termination proofs often need. For example, $n - k < n$ is directly linear and omega solves it outright. But $n \% m < m$ requires apply Nat.mod_lt first (% is outside omega's fragment), then omega handles the residual goal $0 < m$.

decide: evaluation

We saw `decide` in Lecture 22. It works on any closed decidable proposition by evaluating the decision procedure:

example : $10 * 20 = 200$:= **by decide**

example : $\neg(3 < 2 \wedge \text{True})$:= **by decide**

Unlike `omega`, `decide` handles nonlinear arithmetic — but only for specific **closed** values (no free variables). Unlike `simp`, it does not rewrite; it evaluates to true or false.

	Variables?	Nonlinear?	Method
<code>simp</code>	yes	yes	rewriting
<code>omega</code>	yes	no	decision procedure
<code>decide</code>	no (closed only)	yes	evaluation
<code>positivity</code>	yes	yes	compositional

Section 4

Part IV: Combining the Tools

Automation in termination proofs

Termination proofs are a natural place to use automation. Recall the default behavior: when you write `termination_by`, Lean tries a built-in tactic called `decreasing_tactic` that uses `simp_wf` followed by `omega`. This handles most cases automatically.

When it fails, you supply `decreasing_by` with your own tactic. Common patterns:

Goal shape	Tactic
$n - k < n$	<code>omega</code>
$n \% m < m$	<code>apply Nat.mod_lt; omega</code>
<code>List.length xs < ...</code>	<code>simp [List.length]; omega</code>
custom measure	<code>unfold, then omega or simp</code>

A worked example: merge sort

```
def merge [Ord α] (as bs : List α) : List α :=  
  match as, bs with  
  | [],      bs      => bs  
  | as,      []      => as  
  | a::as, b::bs =>  
    if Ord.compare a b |>.isLE  
    then a :: merge as (b::bs)  
    else b :: merge (a::as) bs  
termination_by as.length + bs.length
```

Each recursive call decreases $|as| + |bs|$ by exactly 1. The termination proof is simple linear arithmetic, so Lean discharges it automatically.

Automation in ordinary proofs

The tactics also compose. Here is a proof from Lecture 19, rewritten with automation:

```
-- Lecture 19 style (step by step):  
theorem zero_add_manual :  $\forall n, 0 + n = n$  := by  
  intro n; induction n with  
    | zero => rfl  
    | succ n ih => rw [Nat.add_succ, ih]  
  
-- With automation:  
theorem zero_add_auto :  $\forall n, 0 + n = n$  := by  
  intro n; omega
```

`omega` closes the goal in one step because $0 + n = n$ is linear arithmetic. Note that the manual version actually needed induction: $0 + n$ does *not* reduce when n is a variable, because Lean defines addition by recursion on the second argument ($n + 0$ reduces, but $0 + n$ does not).

Choosing the right tactic

Goal	Try this
$0 + n = n, a < b + 1$	<code>omega</code>
<code>List.length (x :: xs) = ...</code>	<code>simp [List.length]</code>
<code>f (g x) = ...</code> with @[simp] lemmas	<code>simp</code>
$37 * 41 = 1517$	<code>decide</code>
cleanup after <code>rw</code> or <code>induction</code>	<code>simp</code>
linear inequality in <code>decreasing_by</code>	<code>omega</code>

When in doubt, try `simp` first. If it does not close the goal, try `omega`. If the goal is a closed ground fact, try `decide`. Use `simp?` to find out exactly which lemmas `simp` needed.

Section 5

Part V: Under the Hood

Well-founded recursion is structural recursion

How does the kernel check that gcd terminates? It does not know about `termination_by`. Instead, the elaborator rewrites the function to take a **shadow argument** — a proof of $\text{Acc } (<) m$ — and recurse structurally on it:

```
def gcd (m n : Nat) (h : Acc (· < ·) m) : Nat :=
  if hm : m = 0 then n
  else
    have hlt : n % m < m :=
      Nat.mod_lt n (Nat.pos_of_ne_zero hm)
    gcd (n % m) m (h.inv hlt)
```

The recursive call passes `h.inv hlt`: it pattern-matches on `h` to extract a proof of $\text{Acc } (<) (n \% m)$, using `hlt` as the key that selects which sub-proof to extract (recall `Acc.inv` from earlier). This extracted proof is a subterm of `h`, so the kernel sees structural recursion on `Acc`.

Why this works: Acc proofs get smaller

A proof of $\text{Acc } (<) 5$ is built from proofs of $\text{Acc } (<) 4$, $\text{Acc } (<) 3$, ..., $\text{Acc } (<) 0$ — because the constructor requires $\forall y, y < 5 \rightarrow \text{Acc } (<) y$. Each call to Acc.inv peels off one layer, yielding a strictly smaller Acc term.

So the kernel needs no new mechanism. It already knows how to check structural recursion on inductive types. The elaborator just translates well-founded recursion *into* structural recursion on Acc.

A complication: matching proofs to produce data

But there is a problem. $\text{Acc } (<) m$ lives in Prop — it is a proof. Yet gcd returns a Nat , which lives in Type . In Lecture 18, we said all proofs of a proposition are equal (proof irrelevance). If proofs are interchangeable, how can we pattern-match on one?

CIC allows matching on proofs, but restricts what the match can *produce*:

Small elimination (default): the result must be in Prop . You can decompose $h : P \wedge Q$ to build $Q \wedge P$ — the result is a proof, so proof irrelevance is preserved.

Large elimination (exception): the result can be in Type . This is dangerous: matching on $h : P \vee Q$ to return 0 for inl and 1 for inr would produce different data from different proofs. CIC forbids this.

When large elimination is safe

CIC grants large elimination when **one constructor** and **no genuinely new data in** Type. With one constructor, the match has one branch — the result cannot depend on which proof you had.

“Genuinely new data” means: not a **parameter** (fixed for the whole type, written before the colon) and not an **index** (determined by the return type, written after the colon).

```
inductive Acc (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :  $\alpha \rightarrow \text{Prop}$  where
--
--      ~~~~~^
--      parameters      index
| intro (x :  $\alpha$ ) (h :  $\forall y, r\ y\ x \rightarrow \text{Acc}\ r\ y$ ) : Acc r x
--      ~~~~~^~~~~~
--      index      genuinely new (but in Prop)  $\square$ 
```

x appears in the return type $\text{Acc}\ r\ x$ — it is the index value, already known. The only new argument is h , which is in Prop. So Acc qualifies.

Two types that do not qualify

The existential $\exists x : \alpha, P x$ has one constructor, but it carries a witness $w : \alpha$ — genuinely new data that is *not* an index. Matching would extract data from a proof:

```
-- ILLEGAL: extracting a Nat from an existence proof  
def bad (h :  $\exists n : \text{Nat}, n > 5$ ) : Nat :=  
  match h with  
  |  $\langle w, \_ \rangle$  => w      -- w is data, not determined by the type
```

$\vee$ has *two* constructors, which immediately disqualifies it — matching would reveal which disjunct was proved.

Structural vs. well-founded: when to use which

	Structural	Well-founded
Termination	by inductive structure	by decreasing measure
Annotation	none needed	<code>termination_by</code>
Proof	none needed	<code>auto</code> or <code>decreasing_by</code>
Unfolding	definitional	requires <code>simp/unfold</code>
Examples	<code>length</code> , <code>map</code>	<code>gcd</code> , <code>div</code> , <code>mergeSort</code>

Structural recursion is preferred when it works: the function unfolds definitionally, and proofs about it are simpler. Use well-founded recursion when the recursive call does not follow the inductive structure of the input.

Exercises

- 1 Write a function `log2 : Nat → Nat` that computes $\lfloor \log_2 n \rfloor$ by repeated division by 2. Supply `termination_by` and verify that Lean accepts it. What happens without it?
- 2 Implement the Collatz function: $f(n) = n/2$ if n is even, $3n + 1$ if n is odd. Can you define `collatzSteps : Nat → Nat` that counts the number of steps to reach 1? What goes wrong with termination?
- 3 Prove $\forall n m : \text{Nat}, n + m = m + n$ using only `omega`.
- 4 Define a function `fib : Nat → Nat` for Fibonacci numbers using well-founded recursion (not structural). You will need a lexicographic measure or a proof that both $n - 1$ and $n - 2$ are less than n .
- 5 Register `simp` lemmas for `List.length` applied to `map` and `filter`. Use them to prove a fact about the length of a mapped list.
- 6 (Harder) Revisit `buildTree` from Lecture 21 and write the complete `decreasing_by` proof that the list gets shorter at each merge step.

What we covered

- ➊ **Structural vs. well-founded recursion:** when each is used and why.
- ➋ **termination_by:** specifying a decreasing measure (single values, pairs, lexicographic order).
- ➌ **decreasing_by:** supplying a custom termination proof when automation fails.
- ➍ **The theory:** `Acc`, `WellFounded`, `WellFoundedRelation` as a type class; well-founded recursion reduces to structural recursion on `Acc` proofs.
- ➎ **simp:** the simplifier, `@[simp]` lemmas, `simp only`, `simp?`, `simp_all`.
- ➏ **omega:** a decision procedure for linear arithmetic.
- ➐ **decide:** evaluation of closed decidable propositions.

Next lecture: axioms and computation.

What happens when we go beyond what CIC gives us constructively? Propositional extensionality, function extensionality, quotient types, and classical choice. Diaconescu's theorem: how choice implies the law of excluded middle. The noncomputable keyword and the boundary between proofs and programs.